



TDD en Java — Exercices Jour 3

Code Hérité, FitNesse & Outils CI

Sommaire

- Rappel & pré-requis
- Niveau 1 — Code hérité : diagnostic
 - Ex 1.1 — Identifier les problèmes
 - Ex 1.2 — Tests de caractérisation
 - Ex 1.3 — Parameterize Constructor
 - Ex 1.4 — Extract and Override
- Niveau 2 — Code hérité : mise sous test
 - Ex 2.1 — Sprout Method
 - Ex 2.2 — Wrap Method
 - Ex 2.3 — Briser les dépendances statiques
 - Ex 2.4 — Refactoring complet d'un module legacy
- Niveau 3 — FitNesse

Rappel & pré-requis

Les exercices du Jour 3 s'appuient sur les acquis des jours précédents :

- JUnit 5 avec `@Test` , `@BeforeEach` , `@Nested` , `@ParameterizedTest`
- Mockito avec `@Mock` , `@InjectMocks` , `when()` , `verify()` , `ArgumentCaptor`
- Maven avec dépendances JUnit 5 et Mockito configurées

Nouvelles dépendances pour le Jour 3 :

```
<!-- JaCoCo – couverture de code -->
<plugin>
  <groupId>org.jacoco</groupId>
  <artifactId>jacoco-maven-plugin</artifactId>
  <version>0.8.11</version>
  <executions>
    <execution><goals><goal>prepare-agent</goal></goals></execution>
    <execution>
      <id>report</id>
      <phase>test</phase>
      <goals><goal>report</goal></goals>
```

Niveau 1 — Code hérité : diagnostic

Objectif : Apprendre à lire, caractériser et préparer le traitement d'un code legacy sans le casser.

Durée estimée : 1h30

Ex 1.1 — Identifier les problèmes

Contexte : Analysez le code legacy ci-dessous et cataloguez ses problèmes **avant** d'écrire la moindre ligne de test ou de code.

Code legacy fourni :

```
public class FactureManager {

    private static FactureManager instance;
    private Connection conn;

    private FactureManager() {
        try {
            conn = DriverManager.getConnection(
                "jdbc:mysql://prod-server:3306/billing", "admin", "P@ssw0rd!");
        } catch (Exception e) {
            e.printStackTrace();
        }
    }

    public static FactureManager getInstance() {
        if (instance == null) instance = new FactureManager();
        return instance;
    }

    public double calcul(String clientId, String type, int qte) {
        double total = 0;
        try {
            Statement st = conn.createStatement();
            ResultSet rs = st.executeQuery(
                "SELECT prix FROM produits WHERE type='" + type + "'"); // ← ?
            while (rs.next()) {
                total = rs.getDouble("prix") * qte;
                if (clientId.startsWith("VIP")) total = total * 0.85; // ← ?
                if (total > 1000) total = total - 50; // ← ?
                Statement st2 = conn.createStatement();
                st2.executeUpdate(
```

Ex 1.2 — Tests de caractérisation

Contexte : Avant de toucher au code legacy, vous devez capturer son comportement actuel. On vous fournit une version simplifiée (sans BDD réelle) pour l'exercice.

Code legacy simplifié (testable isolément) :

```
public class LegacyCalculeur {  
  
    public String formaterMontant(double montant, String devise) {  
        if (devise == null) devise = "";  
        String s = String.valueOf((int)(montant * 100) / 100.0);  
        if (s.endsWith(".0")) s = s.substring(0, s.length() - 2);  
        return devise + s;  
    }  
  
    public double appliquerRegles(double base, String typeClient, int anciennete) {  
        double result = base;  
        if (typeClient != null && typeClient.equals("PREMIUM")) {  
            result = result * 0.9;  
        }  
        if (anciennete > 5) {  
            result = result - (result * 0.05);  
        }  
        if (anciennete > 10) {  
            result = result - 20;  
        }  
        if (result < 0) result = 0;  
        return result;  
    }  
}
```

Ex 1.3 — Parameterize Constructor

Contexte : Appliquez la technique "Parameterize Constructor" pour rendre le LegacyService ci-dessous testable.

Avant (non testable) :

```
public class RapportLegacy {  
  
    private final DatabaseAccessor db;  
    private final FileWriter fileWriter;  
    private final EmailSender emailSender;  
  
    public RapportLegacy() {  
        // Dépendances créées en dur → impossible à tester  
        this.db = new MySQLDatabaseAccessor("jdbc:mysql://prod:3306/db");  
        this.fileWriter = new LocalFileWriter("/var/reports/");  
        this.emailSender = new SMTPEmailSender("smtp.prod.com", 587);  
    }  
  
    public void genererEtEnvoyer(String mois) {  
        List<Ligne> donnees = db.requeter("SELECT * FROM ventes WHERE mois='" + mois + "'");  
        String contenu = formaterRapport(donnees);  
        String chemin = fileWriter.ecrire("rapport_" + mois + ".txt", contenu);  
        emailSender.envoyer("direction@entreprise.com",  
            "Rapport " + mois, "Voir pièce jointe : " + chemin);  
    }  
}
```

Ex 1.4 — Extract and Override

Contexte : Rendez la classe `ReportService` testable via la technique "Extract and Override" sans modifier l'API publique.

Code legacy :

```
public class WeatherReportService {

    public String genererBulletin(String ville) {
        // Appel API externe – non mockable directement
        HttpClient client = HttpClient.newHttpClient();
        HttpRequest request = HttpRequest.newBuilder()
            .uri(URI.create("https://api.meteo.fr/v2/current?ville=" + ville))
            .build();

        try {
            HttpResponse<String> response = client.send(
                request, HttpResponse.BodyHandlers.ofString());
            String json = response.body();
            return traiterReponse(json, ville);
        } catch (Exception e) {
            return "Données météo indisponibles pour " + ville;
        }
    }

    private String traiterReponse(String json, String ville) {
        // Parsing JSON simplifié
        if (json.contains("\"temp\":")) {
            int start = json.indexOf("\"temp\":") + 7;
            // ... (le reste du code est coupé dans l'image)
```


Niveau 2 — Code hérité : mise sous test

Objectif : Appliquer les techniques avancées de remise sous test sur du code legacy réaliste.

Durée estimée : 2h00

Ex 2.1 — Sprout Method

Contexte : Ajoutez une nouvelle fonctionnalité à une grosse méthode legacy **sans la modifier**, en utilisant la technique "Sprout Method".

Code legacy (ne pas modifier le corps de `processerCommande`) :

```
public class LegacyOrderProcessor {  
  
    public String processerCommande(String clientId, String[] articles, double montantTotal) {  
        // ⚠ 80 lignes de code legacy non testable ici  
        // Accès BDD, logique métier, formatage, envoi email...  
        // ... (simulé par le commentaire)  
  
        // Résultat final  
        String numeroCommande = "CMD-" + System.currentTimeMillis();  
  
        // NOUVEAU BESOIN : appliquer les points de fidélité  
        // Cette logique doit être ajoutée ici  
        // appliquerPointsFidelite(clientId, montantTotal); ← à sprouter  
  
        return numeroCommande;  
    }  
}
```

Ex 2.2 — Wrap Method

Contexte : Ajoutez de la journalisation à une méthode existante sans la modifier, en utilisant "Wrap Method".

Méthode existante (ne pas modifier) :

```
public class PaiementService {  
  
    private final BanqueAPI banqueAPI;  
  
    public PaiementService(BanqueAPI banqueAPI) {  
        this.banqueAPI = banqueAPI;  
    }  
  
    // Méthode existante – ne pas modifier  
    public ResultatPaiement effectuerPaiement(String carteId, double montant) {  
        if (montant <= 0) throw new IllegalArgumentException("Montant invalide");  
        return banqueAPI.debiter(carteId, montant);  
    }  
}
```

Ex 2.3 — Briser les dépendances statiques

Contexte : La classe ci-dessous utilise des appels statiques impossibles à mocker. Rendez-la testable.

Code legacy avec dépendances statiques :

```
public class SessionManager {

    public boolean authentifier(String username, String password) {
        // Appel statique à une classe utilitaire non mockable
        String hash = CryptoUtils.hashSHA256(password);
        User user = DatabasePool.getConnection()
            .query("SELECT * FROM users WHERE login=?", username)
            .first(User.class);

        if (user == null || !user.getPasswordHash().equals(hash)) {
            AuditLog.enregistrer("ECHEC_AUTH", username, LocalDateTime.now());
            return false;
        }

        String token = TokenFactory.generer(user.getId(), 3600);
        SessionStore.setToken(user.getId(), token, user);
    }
}
```

Ex 2.4 — Refactoring complet d'un module legacy

Contexte : Refactorisez complètement le module `FactureManager` de l'Ex 1.1 en suivant le processus complet.

Processus à respecter (étapes séquentielles) :

ÉTAPE 1 – Tests de caractérisation (avec mock de la BDD)

Écrire 5 tests qui documentent le comportement actuel

ÉTAPE 2 – Extraction des interfaces

Créer : `QueryExecutor`, `FactureWriter`, `RemiseCalculateur`

ÉTAPE 3 – Parameterize Constructor

Ajouter le constructeur avec injection (garder l'ancien)

→ Vérifier que les tests de caractérisation passent encore

ÉTAPE 4 – Nommage et constantes

Renommer méthodes et variables (tests toujours verts)

Extraire les constantes (`0.85` → `REMISE_VIP`, etc.)

ÉTAPE 5 – Séparation des responsabilités

Extraire `RemiseService`, `FactureRepository`

Niveau 3 — FitNesse

Objectif : Installer FitNesse, créer des tables de test et les relier à des fixtures Java.

Durée estimée : 1h45

Ex 3.1 — Installation et premier test

Étape 1 — Installation

```
# Créer le dossier de travail
mkdir tp-fitnessse && cd tp-fitnessse

# Télécharger FitNesse standalone
wget https://github.com/unclebob/fitnessse/releases/download/v20231103/fitnessse-standalone.jar

# Lancer FitNesse sur le port 8080
java -jar fitnessse-standalone.jar -p 8080 -d .

# Vérifier l'accès
# Navigateur → http://localhost:8080
```

Étape 2 — Créer votre première page de test

Dans le navigateur FitNesse :

1. Allez sur `http://localhost:8080`
2. Cliquez sur "Edit" sur la page d'accueil

Ex 3.2 — Tables ColumnFixture

Contexte : Créez des tables FitNesse pour tester le `FactureService` refactorisé.

Page FitNesse `MaSuite.TestCalculFacture` :

```
!define TEST_SYSTEM {slim}
!path target/classes
!path target/test-classes
!path target/dependency/*.jar

|import
|com.formation.tdd.fixtures

---- Calcul TVA ----

!! ColumnFixture : CalculTVAFixture |
| montantHT | tauxTVA | tva? | ttc? |
| 100.00    | 0.20    | 20.00 | 120.00 |
| 200.00    | 0.10    | 20.00 | 220.00 |
| 500.00    | 0.055   | 27.50 | 527.50 |
| 0.00      | 0.20    | 0.00  | 0.00   |

---- Calcul avec remise ----

!! ColumnFixture : CalculRemiseFixture |
| montantHT | tauxRemise | montantApresRemise? |
| 100.00    | 0.10       | 90.00                |
| 200.00    | 0.20       | 160.00               |
| 500.00    | 0.50       | 250.00               |
| 100.00    | 0.00       | 100.00               |

---- Calcul TTC avec remise ----
```


Ex 3.3 — Script Tables et scénarios

Contexte : Utilisez les Script Tables pour tester un flux complet de gestion de compte.

Page FitNesse **MaSuite.TestCompteBancaire** :

```
!define TEST_SYSTEM {slim}
!path target/classes
!path target/test-classes
!path target/dependency/*.jar

|import                               |
|com.formation.tdd.fixtures          |

---- Scénario 1 : Dépôts et retraits successifs ----

|script | CompteBancaireFixture | |
|créer compte avec solde initial | 500.00 |
|check  | solde                  | 500.00 |
|deposer | 200.00                 |         |
|check  | solde                  | 700.00 |
|retirer | 150.00                 |         |
|check  | solde                  | 550.00 |

---- Scénario 2 : Retrait impossible ----

|script | CompteBancaireFixture | | |
|créer compte avec solde initial | 100.00 |
|check  | peutRetirer           | 200.00 | false |
|check  | peutRetirer           | 100.00 | true  |
|check  | peutRetirer           | 0.01  | true  |

---- Scénario 3 : Historique ----

|script | CompteBancaireFixture |
|créer compte avec solde initial | 0.00 |
```

Ex 3.4 — Suite FitNesse et intégration CI

Contexte : Organisez vos tests FitNesse en suite et intégrez-les dans Maven.

Étape 1 — Structure de suite

Créez la hiérarchie suivante dans FitNesse :

```
FormationTDD (Suite)
├── SetUp                ← configuration commune
├── TestCalculFacture    ← Ex 3.2
├── TestCompteBancaire   ← Ex 3.3
├── TestGestionStock     ← à créer
└── TearDown            ← nettoyage
```

Page SetUp (configuration partagée) :

```
!define TEST_SYSTEM {slim}
!path target/classes
!path target/test-classes
!path target/dependency/*.jar
```

Niveau 4 — Outillage CI complet

Objectif : Configurer un pipeline CI complet avec tests, couverture et qualité de code.

Durée estimée : 1h30

Ex 4.1 — Pipeline Jenkins/GitHub Actions

Contexte : Créez un pipeline CI pour votre projet de formation.

Option A — GitHub Actions

Créez le fichier `.github/workflows/ci-tdd.yml` :

```
name: CI - Formation TDD Java
on:
  push:
    branches: [ main, develop, 'feature/**' ]
    pull_request:
      branches: [ main ]
  env:
    JAVA_VERSION: '17'
    MAVEN_OPTS: '-Xmx1024m'
jobs:
  # Job 1 : Tests unitaires rapides
  tests-unitaires:
    name: Tests Unitaires
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - name: Setup JDK ${{ env.JAVA_VERSION }}
        uses: actions/setup-java@v4
        with:
          java-version: ${{ env.JAVA_VERSION }}
          distribution: 'temurin'
          cache: maven
      - name: Compilation
        run: mvn compile test-compile --batch-mode
      - name: Tests unitaires + couverture
        run: mvn test --batch-mode
      - name: Publier rapport JUnit
        uses: dorny/test-reporter@v1
        if: always()
        with:
          name: Résultats Tests Unitaires
          path: target/surefire-reports/*.xml
          reporter: java-junit
          fail-on-error: true
      - name: Upload rapport JaCoCo
        uses: actions/upload-artifact@v4
        with:
          name: jacoco-report
          path: target/site/jacoco/
  # Job 2 : Analyse qualité (dépend du job 1)
  qualite:
    name: Analyse Qualité
    runs-on: ubuntu-latest
    needs: tests-unitaires
    steps:
      - uses: actions/checkout@v4
        with:
          fetch-depth: 0 # nécessaire pour SonarQube
      - name: Setup JDK
        uses: actions/setup-java@v4
        with:
          java-version: ${{ env.JAVA_VERSION }}
          distribution: 'temurin'
          cache: maven
  # TODO : Compléter avec l'analyse SonarCloud
  - name: Analyse SonarCloud
    env:
```

Ex 4.2 — SonarQube et Quality Gate

Contexte : Configurez SonarQube et définissez un Quality Gate adapté à votre projet.

Étape 1 — Lancer SonarQube avec Docker

```
# Lancer SonarQube Community Edition
docker run -d \
  --name sonarqube-formation \
  -p 9000:9000 \
  -e SONAR_ES_BOOTSTRAP_CHECKS_DISABLE=true \
  sonarqube:10-community

# Attendre que SonarQube soit prêt (~1-2 minutes)
docker logs -f sonarqube-formation | grep "SonarQube is operational"

# Accès : http://localhost:9000
# Login : admin / admin (changer au premier accès)
```

Étape 2 — Configurer le projet

Niveau 5 — Projet final intégré

Objectif : Combiner toutes les pratiques de la formation sur un projet complet de bout en bout.

Durée estimée : 4h00 (exercice de synthèse)

Ex 5.1 — Bibliothèque numérique

Contexte : Vous rejoignez une équipe qui maintient un système de gestion de bibliothèque numérique. La moitié du code est legacy (sans tests), l'autre moitié doit être développée en TDD.

Architecture existante (legacy — à mettre sous test) :

```
// ⚠ Code legacy – ne pas modifier avant de le couvrir de tests
public class LegacyBibliothequeService {

    private static Connection conn;

    static {
        try {
            conn = DriverManager.getConnection("jdbc:h2:mem:biblio");
        } catch (Exception e) { throw new RuntimeException(e); }
    }

    public boolean emprunterLivres(String isbn, String userId) {
        try {
            ResultSet rs = conn.createStatement().executeQuery(
                "SELECT * FROM livres WHERE isbn='" + isbn + "'");
            if (!rs.next()) return false;
            if (rs.getBoolean("emprunte")) return false;
            conn.createStatement().executeUpdate(
                "UPDATE livres SET emprunte=true, emprunteur='" +
                userId + "' WHERE isbn='" + isbn + "'");
        }
    }
}
```

Corrigés partiels

Corrigé — Ex 1.1 (grille complète)

#	Problème	Catégorie	Impact	Correction
1	Singleton mutable	Couplage	Non testable	Injection dépendances
2	BDD dans constructeur	Architecture	Effets de bord	Parameterize Constructor
3	Credentials en dur	Sécurité	Fuite secrets	Variables d'environnement
4	Injection SQL	Sécurité	Vulnérabilité critique	PreparedStatement
5	catch vide	Robustesse	Bugs silencieux	Relancer ou logger
6	SQL + métier mélangés	SRP	Non maintenable	Séparation en couches
7	Constantes	Lisibilité	Non compréhensible	Constantes nommées

Corrigé — Ex 1.2 (résultats `appliquerRegles`)

base	typeClient	anciennete	Résultat
100.0	"PREMIUM"	3	90.0 (remise VIP seule)
100.0	"STANDARD"	6	95.0 (remise ancienneté seule)
100.0	"PREMIUM"	11	66.35 (VIP × ancienneté × -20)
100.0	null	0	100.0 (aucune remise)
10.0	"PREMIUM"	11	0.0 (résultat négatif plafonné à 0)

Corrigé — Ex 2.1 (amorce Sprout Method)

```
// Nouvelle méthode ajoutée en TDD – ne touche pas au code legacy
void appliquerPointsFidelite(String clientId, double montant) {
    int pointsBase = Math.max(1, (int) montant);

    int multiplicateur = 1;
    if (clientId.startsWith("G-")) multiplicateur = 2;
    else if (clientId.startsWith("P-")) multiplicateur = 3;

    int pointsTotal = pointsBase * multiplicateur;
    fideliteRepository.ajouterPoints(clientId, pointsTotal);
}

// Tests correspondants
@Test
void appliquerPoints_ClientStandard_1PointParEuro() {
    appliquerPointsFidelite("STD-001", 50.0);
    verify(fideliteRepository).ajouterPoints("STD-001", 50);
}

@Test
void appliquerPoints_MontantFaible_MinimumUnPoint() {
    appliquerPointsFidelite("STD-001", 0.50);
    verify(fideliteRepository).ajouterPoints("STD-001", 1);
}
```

Corrigé — Ex 3.2 (fixture Java complète)

```
public class CalculTVAFixture {  
    public double montantHT;  
    public double tauxTVA;  
  
    private final CalculateurTVA calculateur = new CalculateurTVA();  
  
    public double tva() {  
        return Math.round(montantHT * tauxTVA * 100.0) / 100.0;  
    }  
  
    public double ttc() {  
        return Math.round((montantHT + tva()) * 100.0) / 100.0;  
    }  
}
```

Exercices Jour 3 — Formation TDD Java · © 2026